Dynatrace SDKs

> **Series:** AUTOM | **Notebook:** 6 of 8 | **Created:** January 2026

Table of Contents

Prerequisites

Before starting this notebook, ensure you have:

Requirement	Description
Node.js 18+	For TypeScript SDK
Python 3.9+	For Python SDK
API Token	Token with required scopes
Development Environment	VS Code or similar IDE

Learning Objectives

By the end of this notebook, you will:

- Understand the Dynatrace SDK architecture
- Know how to use TypeScript and Python clients
- Be able to query data and manage configurations programmatically
- Build custom automation applications

---

1. Introduction

Dynatrace provides official SDKs for programmatic access to the platform. These SDKs are auto-generated from OpenAPI specifications, ensuring they stay current with API changes.

Available SDKs

SDK	Language	Use Case
@dynatrace-sdk	TypeScript/JavaScript	Web apps, Node.js tools
dt-sdk	Python	Scripts, data science

SDK vs Direct API

Aspect	SDK	Direct API
Type safety	Yes	No
Auto-completion	Yes	No
Authentication	Built-in	Manual
Pagination	Handled	Manual
Error handling	Structured	Raw HTTP

Client Libraries

Client	Purpose
QueryClient	Execute DQL queries
SettingsClient	Manage Settings 2.0 objects
EntitiesClient	Query entity topology
MetricsClient	Query and ingest metrics
EventsClient	Query and ingest events
LogsClient	Query and ingest logs

---

2. TypeScript SDK

Installation

```
npm install @dynatrace-sdk/client-query
npm install @dynatrace-sdk/client-classic-environment-v2
npm install @dynatrace-sdk/client-settings-v2
```

Configuration

```
import { setEnvConfig } from '@dynatrace-sdk/client-core';

// Configure environment
setEnvConfig({
  environmentUrl: process.env.DT_TENANT_URL,
  apiToken: process.env.DT_API_TOKEN
});
```

Query Example

```
import { queryClient } from '@dynatrace-sdk/client-query';

async function queryHosts() {
  const result = await queryClient.query({
    body: {
      query: `
        fetch dt.entity.host
        | fieldsAdd name = entity.name, osType
        | sort name asc
        | limit 10
      `,
      defaultTimeframeStart: 'now-24h',
      defaultTimeframeEnd: 'now'
    }
  });

  console.log('Hosts:', result.result?.records);
  return result.result?.records;
}
```

Settings Management

```
import { settingsObjectsClient } from '@dynatrace-sdk/client-classic-
environment-v2';

// List management zones
async function listManagementZones() {
  const response = await settingsObjectsClient.getSettingsObjects({
    schemaIds: 'builtin:management-zones',
    pageSize: 100
  });

  return response.items;
}

// Create management zone
async function createManagementZone(name: string) {
  const response = await settingsObjectsClient.postSettingsObjects({
    body: [{
      schemaId: 'builtin:management-zones',
      scope: 'environment',
      value: {
        name: name,
        rules: []
      }
    }]
  });

  return response;
}
```

Entity Queries

```
import { entitiesClient } from '@dynatrace-sdk/client-classic-environment-v2';

async function getServices() {
  const response = await entitiesClient.getEntities({
    entitySelector: 'type(SERVICE)',
    fields: '+properties,+tags',
    pageSize: 100
  });

  return response.entities;
}
```

---

3. Python SDK

Installation

```
pip install dt-sdk
```

Configuration

```
import os
from dynatrace_sdk import DynatraceClient

# Initialize client
client = DynatraceClient(
    environment_url=os.environ['DT_TENANT_URL'],
    api_token=os.environ['DT_API_TOKEN']
)
```

Query Example

```
from dynatrace_sdk.query import QueryService

def query_hosts():
    query_service = QueryService(client)

    result = query_service.query(
        query="""
            fetch dt.entity.host
            | fieldsAdd name = entity.name, osType
            | sort name asc
            | limit 10
        """,
        default_timeframe_start='now-24h',
        default_timeframe_end='now'
    )

    return result.records
```

Settings Management

```python
from dynatrace_sdk.settings import SettingsService

def list_management_zones():
    settings = SettingsService(client)

    zones = settings.list_objects(
        schema_ids='builtin:management-zones',
        page_size=100
    )

    return zones

def create_management_zone(name: str):
    settings = SettingsService(client)

    result = settings.create_object(
        schema_id='builtin:management-zones',
        scope='environment',
        value={
            'name': name,
            'rules': []
        }
    )

    return result
```

Pagination Handling

```
def get_all_hosts():
    """Iterate through all pages of hosts."""
    all_hosts = []
    next_page_key = None

    while True:
        response = client.entities.get_entities(
            entity_selector='type(HOST)',
            page_size=500,
            next_page_key=next_page_key
        )

        all_hosts.extend(response.entities)

        if not response.next_page_key:
            break
        next_page_key = response.next_page_key

    return all_hosts
```

4. Common Patterns

Bulk Configuration Export

```
// TypeScript: Export all settings for backup
import { settingsObjectsClient, settingsSchemasClient } from '@dynatrace-
sdk/client-classic-environment-v2';
import * as fs from 'fs';

async function exportAllSettings() {
  // Get all available schemas
  const schemas = await
settingsSchemasClient.getAvailableSchemaDefinitions();

  const export_data: Record&lt;string, any[]&gt; = {};

  for (const schema of schemas.items || []) {
    const objects = await settingsObjectsClient.getSettingsObjects({
      schemaIds: schema.schemaId,
      pageSize: 500
    });

    if (objects.items &amp;&amp; objects.items.length &gt; 0) {
      export_data[schema.schemaId] = objects.items;
    }
  }

  fs.writeFileSync('settings-export.json', JSON.stringify(export_data, null,
2));
  return export_data;
}
```

Error Handling

```
import { ApiError } from '@dynatrace-sdk/client-core';

async function safeQuery(query: string) {
  try {
    const result = await queryClient.query({
      body: { query }
    });
    return { success: true, data: result };
  } catch (error) {
    if (error instanceof ApiError) {
      console.error(`API Error: ${error.status} - ${error.message}`);
      return { success: false, error: error.message };
    }
    throw error;
  }
}
```

Rate Limiting

```
import time
from functools import wraps

def rate_limited(max_per_second: float):
    """Decorator to rate limit API calls."""
    min_interval = 1.0 / max_per_second
    last_called = [0.0]

    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            elapsed = time.time() - last_called[0]
            if elapsed &lt; min_interval:
                time.sleep(min_interval - elapsed)
            result = func(*args, **kwargs)
            last_called[0] = time.time()
            return result
        return wrapper
    return decorator

@rate_limited(10)  # Max 10 requests per second
def api_call(entity_id: str):
    return client.entities.get_entity(entity_id=entity_id)
```

Async Operations

```typescript
// TypeScript: Concurrent queries with rate limiting
import pLimit from 'p-limit';

const limit = pLimit(5);  // Max 5 concurrent requests

async function queryMultipleHosts(hostIds: string[]) {
  const queries = hostIds.map(hostId =&gt;
    limit(() =&gt; queryClient.query({
      body: {
        query: `
          fetch dt.entity.host
          | filter id == "${hostId}"
        `
      }
    }))
  );

  return Promise.all(queries);
}
```

5. Building Applications

CLI Tool Example

```
#!/usr/bin/env ts-node
import { setEnvConfig } from '@dynatrace-sdk/client-core';
import { queryClient } from '@dynatrace-sdk/client-query';
import { Command } from 'commander';

const program = new Command();

program
  .name('dt-cli')
  .description('Dynatrace CLI tool')
  .version('1.0.0');

program
  .command('hosts')
  .description('List all hosts')
  .action(async () =&gt; {
    setEnvConfig({
      environmentUrl: process.env.DT_TENANT_URL!,
      apiToken: process.env.DT_API_TOKEN!
    });

    const result = await queryClient.query({
      body: {
        query: 'fetch dt.entity.host | fields entity.name'
      }
    });

    console.table(result.result?.records);
  });

program.parse();
```

Reporting Script

```
import pandas as pd
from dynatrace_sdk import DynatraceClient
from dynatrace_sdk.query import QueryService

def generate_host_report():
    """Generate a host inventory report."""
    client = DynatraceClient(
        environment_url=os.environ['DT_TENANT_URL'],
        api_token=os.environ['DT_API_TOKEN']
    )

    query_service = QueryService(client)

    result = query_service.query(
        query="""
            fetch dt.entity.host
            | fieldsAdd
                name = entity.name,
                osType,
                cpuCores = entity.detected.properties[`host.cpuCores`],
                memoryTotal = entity.detected.properties[`host.memoryTotal`]
            | sort name asc
        """
    )

    df = pd.DataFrame(result.records)
    df.to_csv('host_inventory.csv', index=False)

    return df
```

Best Practices

Practice	Description
Environment variables	Never hardcode credentials
Type safety	Use TypeScript types or Python type hints
Error handling	Catch and handle API errors
Pagination	Always handle paginated responses
Rate limiting	Respect API limits
Logging	Add structured logging

---

6. Next Steps

When to Use SDKs

Scenario	Tool Choice
Custom reporting tool	SDK
One-off query	Direct API or Notebooks
Config deployment	Monaco or Terraform
Integration app	SDK
Complex logic	SDK with JavaScript/Python

Continue the Series

Next Notebook	Focus
AUTOM-07: CI/CD Integration	GitOps patterns and pipelines

Additional Resources

- Dynatrace Developer Portal
- TypeScript SDK Documentation
- Python SDK on PyPI
- API Reference

Summary

In this notebook, you learned:

- How to use the TypeScript and Python SDKs
- Common patterns for queries, settings, and entities
- Building custom CLI tools and reporting scripts
- Best practices for SDK usage

> **Key Takeaway:** SDKs provide the most flexibility for custom applications. Use them when you need programmatic access, complex logic, or integration with other systems. For simple config management, Monaco or Terraform are often simpler.

Continue to AUTOM-07: CI/CD Integration to learn GitOps patterns.
